

```
t2: 3, 9
t1: 5, 10
```

Recall that the second number on each line is the content of our static counter. It seems a little odd that we've skipped over counter == 6. How could have this happened? Keep in mind that counter++ is really the following (conceptual) set of instructions:

```
int temp = counter;
counter = counter + 1;
return temp;
```

Also recall threads are scheduled by the runtime/OS at will. So the following "bad" scheduling can happen:

```
t2: pulls the value 1 from the counter and stores it
in temp
t1: pulls the value 5 from counter and stores it in
temp
t1: increments counter to 6
t1: prints "t1: 4, 5"
t2: increments counter to 7
t2: prints "t2: 0, 1"
```

This is an example of a race condition between threads. We would like the counter++ set of instructions to execute all at once without being pre-empted. However, the runtime/OS is free to schedule threads any way it sees fit, thus leading to the anomaly.

Clearly this is just a toy example, but imagine if counter was not a simple counter but a bank account and the competing threads were the bank account owner and the bank doing operations on the account. In this case, race conditions like above are not acceptable; we need some way to ensure that only one thread "accesses" counter at a time.

C# provides this functionality through its locking mechanism analogous to Java's synchronized construct.

```
public class LockingThreadExample
{
    public static void Test()
    {
        Thread t1 = new Thread(new ThreadStart(PrintSeq));
        t1.Name = "t1";
        Thread t2 = new Thread(new ThreadStart(PrintSeq));
        t2.Name = "t2";
```